

Plan tehnic · OCR hibrid

PaddleOCR + TrOCR (scris de mână)

Transitix · Companion RAI Documente

Branch: **demo-experimenting-RAISpedition** · Sidecar: **tools/paddle-ocr** (FastAPI, port 8100)

Document intern pentru asociați · 23 septembrie 2026 · revizie: watchlist implementare (STOP flaky, pin tokenizers, HYBRID=0 vs baseline)

VM 4 GB RAM

MEMORY_PROFILE=tight

Fără rescriere Node.js

Baseline = același set

Cuprins

1. Problema și obiectivul
2. Constrângeri
3. Soluția propusă
4. Ordinea de lucru (gate-uri)
5. Pipeline tehnic
6. Configurare (environment)
7. Livrabile și pași
8. Ce nu facem

1. Problema și obiectivul

Sidecar-ul OCR actual folosește PaddleOCR cu `latin_PP-OCrv3_mobile_rec`, antrenat pe **text tipărit**. Pe scris de mână, confidența cade des la **0,1–0,3** și textul e greu de folosit în facturare.

Obiectiv

- **Tipărit:** Paddle + fallback când câmpurile esențiale lipsesc / conf slabă.
- **Scris de mână:** TrOCR-small ONNX pe CPU + fallback controlat.
- Acuratețe maximă pe demo, consum minim RAM/VRAM.

Hardware demo (self-hosted)

- **4 GB RAM** (strâns) · GTX **1060 Ti** · Ryzen 7 4800H
- Inferență **CPU** în container (GPU nefolosit în v1)
- Profil `MEMORY_PROFILE=tight`: 1 request, DET_SIDE 1280, TrOCR lazy+unload, clasificator heuristic

2. Constrângeri

- Node.js neschimbat — același `POST /ocr / /ocr/json` pe port **8100**.
- Preprocess existent păstrat (perspectivă, auto-rotate, `DET_SIDE_LEN=1920`, `DROP_SCORE=0.10`).
- Fără Qwen / modele > 1B / PaddleOCR-VL / llama.cpp.
- Câmpurile de business rămân în Node; sidecar adaugă metadata (`needs_review`, linii, stats).

Optimizare pe 4 GB: un OCR o dată; `DET_SIDE=1280`; fără MobileNet în RAM (heuristică); TrOCR lazy + unload după request; ORT 1–2 thread-uri; pass-uri foto limitate; RSS în `/health`. Dacă baseline (doar Paddle) deja OOM pe VM → nu adăuga TrOCR pe aceeași cutie fără swap sau mutare sidecar. Implementarea **nu pornește** până comanda explicită „începe”.

3. Soluția propusă

- 1. PaddleOCR detectează zonele de text.
- 2. Clasificator ușor (MobileNetV3 ONNX / heuristică CPU): printed vs handwritten.
- 3. Printed → Paddle; handwritten → TrOCR-small ONNX.
- 4. Fallback-uri pe praguri de confidență / lungime.
- 5. Validare câmpuri desemnate + `needs_review`.
- 6. Măsurători: zone Paddle vs TrOCR, timp, documente în review.

TrOCR-small ONNX: consum minim, potrivit pentru demo. Export cu *greedy decoding* (fără beam search) — acceptabil pe demo. **Vocabular BPE obligatoriu** la export; pin-uieste `tokenizers` în `requirements.txt` la aceeași versiune ca la export (mismatch = text stricat / crash).

4. Ordinea de lucru (gate-uri obligatorii)



GATE 0 — ÎNAINTE DE ORICE COD HYBRID

10–20 avize reale (anonimizate) prin pipeline-ul *actual*. Raportul include câmpuri lipsă, timp și **confidențele pe linii** (chiar 0,1–0,3 pe HW — necesare la comparație). Lista de fișiere este **înghețată**: primul test hybrid rulează pe **exact același set**. Fără raport pe disc → fără cod hybrid.

GATE 1 — PROBE AMBELE CĂI PADDLE 2.9

`probe_det_rec.py` rulează **întotdeauna ambele**: (A) `ocr(det/rec flags)` și (B) `text_detector` + `text_recognizer`. Chiar dacă A „pare OK”, B se testează — pe 2.9 A poate fi inconsistentă. Pipeline-ul folosește doar calea *stabilă*. Dacă **ambele** căi sunt flaky: **STOP** — investighează versiunea PaddleOCR / modelul `latin_PP-OCrv3_mobile_rec`; nu forța pipeline-ul pe fundație instabilă.

4b. Watchlist la implementare

Moment	Ce verifici	Dacă eșuează
GATE 1	Căile A+B stabile pe 2–3 rerun-uri	STOP — Paddle / model latin; nu forța pipeline
Export TrOCR	Vocab BPE + aceeași versiune <code>tokenizers</code> pin-uită (export = container)	Re-export / aliniere pin
Primul run pe set	Mai întâi <code>HYBRID_OCR=0</code> vs baseline; apoi <code>=1</code>	Dacă <code>=0</code> ≠ baseline → regresie refactor; repară înainte de a judeca hybrid
Re-bench <code>HYBRID_OCR=1</code>	Distribuția confidențelor pe linii HW , nu doar <code>missing_fields</code>	Missing similar + conf HW ↑ (ex. 0,1–0,3 → 0,7–0,9) = TrOCR funcționează; nu judeca doar după câmpuri lipsă

Moment	Ce verifici	Dacă eșuează
field_check	2–3 docs cu <code>paddle_fallback</code> : text slab inclus în validare	Fix field_check — nu exclude fallback

5. Pipeline tehnic

1. Preprocess existent → detecție (API din Gate 1).
2. Clasificare printed / handwritten per crop.
3. Printed → Paddle; dacă conf < **0,5** → TrOCR.
4. Handwritten → TrOCR la **384×384** (resize + padding, fără stretch). Crop-uri foarte late (aspect > 10:1, ex. 2000×50): padding centrat umple prea mult cu alb — preferabil **tăiere în bucăți** sau padding diferit (notă în `trocr_onnx.py`). Conf < **0,75** sau lungime < **2** → `paddle_fallback`.
5. Agregare + `field_check` + răspuns extins (Node citește în continuare `text`).

`paddle_fallback` : text slab, dar *prezent cu încredere scăzută* — nu se exclude din validare (altfel needs_review pe aproape tot).

Ruta / adresa lungă (HW): nu blochează demo-ul. (1) Split TrOCR pe crop-uri late (aspect >10:1); (2) `OPTIONAL_FIELDS=ruta,adresa` — warning, nu `missing_fields`; (3) lexicon fuzzy pe localități/străzi (`lexicon/ro_places.txt`). Ținta client: TPO, dată, nr. auto, TRO, cantitate. Adresa poate rămâne „de confirmat”.

Observabilitate

- `GET /health` : contoare + `hybrid: true|false` din `HYBRID_OCR`.
- `POST /debug/classify` : crop → stil + confidență.
- ONNX versionat: `trocr-small-handwritten-v1.onnx` + vocab + sha256.

6. Configurare (environment)

Variabilă	Default	Rol
<code>HYBRID_OCR</code>	1	0 = doar Paddle; health: <code>hybrid: false</code>
<code>PRINTED_FALLBACK_CONF</code>	0.5	Sub prag → TrOCR pe crop printed
<code>HANDWRITING_THRESHOLD</code>	0.75	Conf minimă TrOCR
<code>HANDWRITING_MIN_LEN</code>	2	Sub → zgomot → <code>paddle_fallback</code>
<code>REQUIRED_FIELDS</code>	aviz, dată, nr. auto, cantitate, semnătură (+ TRO)	fără ruta/adresa
<code>OPTIONAL_FIELDS</code>	ruta, adresa	warning / low_conf, nu blocker
<code>TROCR_ONNX</code> / <code>CLASSIFIER_ONNX</code>	...-v1.onnx	+ vocab BPE pentru TrOCR

7. Livrabile și pași

#	Pas	Rezultat
0	Baseline 10–20 avize (set înghețat + confidențe)	Raport pe disc
1	Probe det/rec — căile A și B	API stabil ales
2	Pipeline + clasificator + TrOCR (+ vocab BPE)	Sidecar hybrid
3	field_check, /health, /debug/classify	Observabilitate
4	Docker / README / teste	Reproductibil
5	Pe același set: HYBRID_OCR=0 vs baseline, apoi =1 — succes HW = conf ↑ pe linii HW	Dovadă TrOCR chiar dacă missing_fields rămâne similar

8. Ce nu facem

- Nu scriem cod hybrid înainte de raportul baseline.
 - Nu oprim probe-ul la prima cale care „pare OK”; dacă ambele flaky → STOP, nu forțăm pipeline.
 - Nu exportăm TrOCR ONNX fără vocab BPE / fără pin tokenizers aliniat.
 - Nu judecăm hybrid-ul dacă HYBRID_OCR=0 deja diferă de baseline.
 - Nu schimbăm setul de avize între baseline și hybrid.
 - Nu stretch-uim TrOCR; strip-uri >10:1 → split / pad special.
 - Nu excludem paddle_fallback din field_check (verificare manuală 2–3 docs).
 - Nu modificăm Node.js / portul sidecar-ului.
 - Nu introducem modele VL grele pe GPU pentru acest demo.
- Mesaj pentru asociați:** extensie a sidecar-ului OCR existent, nu rescriere a aplicației. Succesul se măsoară pe *același* set de documente reale — baseline vs hybrid — inclusiv confidențe.